

Towards walkable footpath detection for the visually impaired on Bangladeshi roads with smartphones using deep edge intelligence

Md. Ishan Arefin Hossain^{*,} Jareen Anjom, Rashik Iram Chowdhury

Department of Electrical and Computer Engineering, North South University, Bashundhara R/A, Dhaka, 1229, Bangladesh

ARTICLE INFO

Keywords:

Walkable footpath
Image segmentation
Edge intelligence
Deep learning
Computer vision

ABSTRACT

One of the ongoing prevalent issues is the challenge faced by visually impaired people when crossing footpaths, especially in a densely populated geographic location such as Dhaka city in Bangladesh, where numerous accidents take place that primarily result in the demise of the affected individuals. Visually impaired people find themselves in precarious situations while navigating through these footpaths. So, having an accessible edge device like a smartphone capable of predicting walkable footpaths by detecting obstacles in real-time is a blessing. However, little work has been done on efficient obstacle detection on footpaths and their corresponding distance prediction in real-time. To address this burning issue, a U-Net-based lightweight deep learning model called QPULM along with an obstacle distance measurement technique called SODD have been proposed in this research, which is utilized in an Android application to detect walkable footpath by avoiding the obstacles via the image captured and to broadcast the directions of the walkable paths using audio feedback. The proposed novel lightweight model at the Edge showed an excellent accuracy of 99.37% with a faster prediction time in milliseconds in real-time, which is significantly better and more efficient than the existing related solutions.

1. Introduction

The visually impaired people face several challenges in their daily lives, especially while trying to navigate their way in public. According to the World Health Organization [1], about 2.2 billion people are visually impaired globally in 2023. The majority of people with sight take the shortest walking path, relying on their natural sense of direction. These abilities rely on environmental information gained via familiarity and perception of the place in which they are situated. The natural skill of spatial orientation that helps one understand one's surroundings is essential when navigating on foot. Spatial orienting primarily depends on the sense of sight and visual cues, such as distinguishable landmarks that can be easily recognized and identified along a route. Tall buildings and towers are some examples of identifiable landmarks. Pedestrians with visual impairment cannot use this ability as they go through a path. Physical impediments and dangers, such as potholes, cracks, stalls, and other pedestrians, make it harder for them to cope. One of the many challenges is providing a sustainable solution that overcomes such limitations and becomes a channel for navigation in a dense environment. This solution is more necessary for people living in Bangladesh, the eighth most densely populated country [2] in the world.

The footpaths of Bangladeshi roads are often barricaded with various kinds of obstacles. Especially in Dhaka, the capital city, the footpaths lack continuity. While walking, one is bound to be obstructed by construction materials, roadside hawkers, an electrical pole, or a tree, as demonstrated in Fig. 1. The numerous street vendors block half the way and litter the street, making walking difficult. Broken pathways, potholes, and crowds also pose the same problem. This hinders pedestrians from going comfortably down a footpath. Pedestrians are frequently observed hopping over maintenance holes, avoiding impediments on the paths just for a journey of a few blocks across the city. All of this elevates walking in Dhaka to the level of extreme sports. People often find themselves in precarious situations, losing their way and stumbling upon obstacles, which is a phenomenon considering how populous the country is. Dhaka has an alarmingly high population density, which is increasing by the day. Furthermore, the footpaths are relatively narrow. This creates a perilous situation in which a person must traverse that stretch of pavement while avoiding colliding with another pedestrian. The visually impaired people face the most challenges while navigating through the footpaths of Bangladesh. About six million Bangladeshis are visually impaired [3], and this number is growing, yet they do not have access to cost-friendly, efficient

* Corresponding author.

E-mail address: ishan.hossain@northsouth.edu (Md.I.A. Hossain).



Fig. 1. Dhaka Footpath with obstructions.

navigation systems. Having a guide to help them walk cautiously is not always feasible considering the restrictions of resources; a walking stick is not the most convenient to detect and guide through the crowd, and an automated wheelchair is costly and not commonly available.

In addressing the limitations of current solutions for footpaths in Bangladesh, a revolutionary approach is found in the application of edge intelligence. The methods mentioned above have proven less robust and adaptable to the unique challenges faced in this context. Edge intelligence, however, stands out as a viable alternative. This methodology involves the deployment of artificial intelligence (AI) directly on edge devices, such as sensors or other smart devices, offering a more resilient and contextually relevant solution. Integrating AI-based solutions on edge devices can mitigate the risk factor involved as the models are trained using real-world data and are quickly adaptable and efficient. The proposed solution using deep edge intelligence can be more systematic and user-friendly in detecting the walkable areas of the footpath ahead, hence avoiding obstacles. This solution provides a lightweight and computationally efficient method to navigate through footpaths easily.

The proposed solution incorporates a polished dataset [4], which has over 3000 footpath images of different areas of Dhaka city, along with the corresponding masked area of the walkable path. Quantized and Pruned UNet-based Lightweight MobileNet Model (QPULM), a lightweight deep learning model, is trained on the dataset. Unstructured iterative pruning and quantization techniques using TensorFlow Lite are used to optimize the model. Unlike existing solutions, the computer vision-based assistance system [5] fails to implement an easy-to-use device such as a smartphone and is overly reliant on sensors.

The proposed method in this paper highlights the following contributions:

- Propose a Quantized and Pruned UNet-based Lightweight MobileNet Model (QPULM) for semantic segmentation of footpaths to detect the walkable versus non-walkable area accurately.
- Introduce a Segmentation based Obstacle Distance Detection (SODD) technique to detect obstacles and their approximate first-person point-of-view distance in the left, right, and forward directions.
- Develop a smartphone application that will capture live images and deploy QPULM and SODD to efficiently deliver audible instructions to the visually impaired on probable distances of obstacles.

The following details are mentioned in the rest of the paper. Section 2 discusses some of the recent related works. The methodology,

which highlights the unique approach entailing the system and a detailed description of the model architecture, has been covered in Section 3. This is followed by a comprehensive report of the results and evaluation in Section 4, and concluding the paper in 5, stating future work as well.

2. Related works

There are limited papers that have directly prioritized computer vision techniques to assist visually impaired people. The following papers are similar in context to road-based obstacle detection, which has utilized such techniques without focusing on optimization or integrating them into edge devices.

The paper by Hui R. Ng et al. [6] proposes a classification method using the LSTM network to analyze gait data gathered from a wearable accelerometer automatically identifying irregular walking surfaces. The LSTM network achieved an AUC of 88%. However, the system can be improved further based on the AUC score. The wearable accelerometer is also not readily accessible to the public. A. Datta et al. [5] used Faster R-CNN in an InceptionV2 coco architecture to analyze the environment around the road and the environment around the car. The network was trained for 19 object classes with an accuracy of 86.42%. The proposed system does not have the best accuracy and lacks the ability to be computationally efficient. K. Swathi et al. [7] devised a hybrid deep learning model using UNet architecture with ResNet for indoor and outdoor navigation. The paper compares the hybrid model with the other two models — ResNet and UNet. The resulting loss accumulated from the hybrid model is above 4, which could be improved.

In the paper by N. Kumar & A. Jain [8], the navigation is done using a vision-based 3D-designed wearable model and a vision-based smart stick. The model was trained using YOLO algorithms to detect the path by a GPS-based smart stick. This model only reaches up to 89.24% accuracy. B. Sophia et al. [9] proposed a new deep learning model PFPN-ADT to detect anomalies in pedestrian walkways such as vehicles and skaters. This model encompasses PFPN-based object detection, Stacked Auto Encoder (SAE) based object classification, and Compact Bat Algorithm-based parameter optimization. The model performed better in comparison to the R-CNN and Faster CNN models. In the study by Hadeel Alsolai et al. [10], a new deep-learning algorithm, SCADL-ADPW, was proposed, which identifies the presence of anomalies on pedestrian walkways using remote sensing images with VGG16 as the feature extraction model.

Irina V. Pustokhina et al. [11] developed an automated DLADT-PW model that detects and classifies anomalies in pedestrian walkways such as cars, skates, and jeeps. Mask R-CNN with baseline DenseNet-169 is applied for the detection process. The UCSD dataset was used to test anomaly detection. The DLADT-PW model achieves the highest accuracy. The research by Manal Abdullah Alohalil et al. [12] proposes the ADPW-FLHHO algorithm on remote sensing images, which focuses on identifying and classifying anomalies such as vehicles on pedestrian walkways. This technique uses the HybridNet model tested on (UCSD) Ped1, UCSD Ped2, and avenue datasets while comparing with other existing models, concluding that this technique is the best-performing automated model. This federated learning approach was used due to its non-data distribution method. The paper by Hussan et al. [13] proposes a real-time object detection and recognition system. The system integrates deep learning methods for object detection and recognition, employing Deep Neural Networks to predict objects from camera inputs. They integrated the Yolov3 algorithm, which resulted in a lower precision of about 46% and is not lightweight due to its large model size.

In light of edge intelligence, some papers have carried out computer vision techniques that have substantially contributed to the proposed system.

Jagdish K. M. et al. [16] presented a novel visual assistance system using deep learning and point cloud processing to perform advanced

Table 1
Comparison of existing works on navigation for visually impaired people.

Ref	Year	Dataset	Techniques	Metrics	Key features	Limitations
[14]	2025	COCO Dataset	YOLOV8	Acc = 91.70%	Object recognition	May have high latency, limited accuracy
[15]	2024	Crowd-sourced Dataset	PSPUNet	–	Sidewalk detection	Not an edge approach, may have high latency
[7]	2021	Collected images	U-Net ResNet	Acc = 93.36%	Segmentation & Object detection	limited to only object detection in traffic environment
[8]	2022	Custom	YOLOv5	Acc = 89.24%, mAP = 0.823	Wearable model and smart stick that detects left, right, and straight path	Accuracy can be further improved, limited environment
[9]	2023	Cityscapes, ADE20k and Vistas	PFPN-ADT	AUC = 94.23%	Anomaly detection on pedestrian walkways	No device implementations
[10]	2023	UCSD	SCADL-ADPW	Acc = 99.24%	Anomaly detection on pedestrian walkways	Heavyweight model, no device implementations
[11]	2021	UCSD	DLADT-PW	Acc = 98.2%	Anomaly detection on pedestrian walkways	Heavyweight model, limited anomalies
[12]	2023	UCSD	ADPW-FLHHO	AUC = 99.36%	Federated learning for anomaly detection	Limited environment
[13]	2022	COCO	YOLOv3	mAP = 46.85%	Object Detection & GTTS for auditory feedback	Limited to generic object detection
[16]	2021	GOI, LISA, GTSRB, Traffic Cone, Cityscapes	SSD MobileNet	N/A	GPU-based hardware, smart depth sensors (OAK-D)	Expensive backpack-based unit
[17]	2023	Custom Object & Pothole Dataset	MobileNet SSD	Acc = 72%	Raspberry Pi, ultrasonic sensors	Small dataset
[18]	2023	COCO	SSDLite MobileNetV2	Acc = 88.89%	Headcap with Raspberry Pi	Not an accessible device for regular use, limited accuracy

perception tasks. The proposed system assesses traffic conditions and the detection and localization of hanging obstacles, crosswalks, moving obstacles, and sudden elevation changes. Many datasets were used to train their model, including their custom dataset. SSD MobileNet was used for object detection, and TrafficSignNet was used to classify traffic signs. A discrete system integrates an OAK-D sensor in a chest-mounted vest, wirelessly relaying AI-driven visual data to Bluetooth-enabled earphones via a backpack-based computing unit, which is less convenient to use due to multiple devices required for application.

V. K N et al. [17] proposed a system using Raspberry Pi, a deep learning model and Pi camera. Images/videos captured using the Pi camera are sent to the MobileNet SSD deep learning model for classification and object detection. An ultrasonic sensor unit, worn on the waist, is used to detect or sense nearby objects, which then sends a signal to the vibration motor to alert the user based on the distance between the object and the user. However, the accuracy achieved is 72%, as the model has been tested using a small dataset and relatively shows slower responsiveness in real-time applications. R. Lima et al. [19] introduce a system where a smartphone is used to process the surroundings of the current location. The proposed solution can analyze pictures in fractions of a second with satisfactory accuracy. An image recognition algorithm is used to identify landmarks. This algorithm uses ORB for feature detection and k-nearest neighbor (KNN) for feature matching. In the research paper by Islam et al. [18], an assistive system uses object detection, created using a pre-trained SSD Lite MobileNetV2 model, trained on the COCO dataset for weather detection. Real-time video is captured using a Raspberry Pi camera, which is integrated into a head cap further connected to earphones for audio feedback using speech recognition. However, the system is not integrated into a mobile device, affecting its usability and engenders an accuracy of 88.89%.

[20] Some methods have used proximity detection systems to detect objects in proximity of the visually impaired person. Using hybridization of IoT and computer vision models, the paper [21] uses ultrasonic sensors embedded in Raspberry Pi to detect object proximity in an

office environment with an accuracy of 98.7%. Another paper [22] uses a sub-scene target detection neural network model for extracting object information of the target or anomaly. The CNN-LSTM model helps detect various scenes, such as urban areas, barren land, vegetation, and water bodies, which can be useful for detecting scene-specific objects. A low-cost assistive device [23] can use a BCM2711 processor and a Pi camera for object identification and live streaming. This device can help the navigation process using a VR headset. However, it is not appropriate for navigating on a day-to-day basis due to the bulkiness of the system. The Sidewalk AI Scanner [15] utilizes crowdsourced and low-cost sidewalk mapping using smartphone cameras through a web app. Although not an edge approach, it proposes a visual AI-based PSPUNet model to detect sidewalks and their width. The paper by Gabriel et al. [14] proposes a smart assistive navigation system that performs object recognition using YOLOV8 implemented on a Raspberry Pi with an accuracy of 91.70%. It also provides auditory feedback on item distance, orientation, and surface wetness.

Table 1 compares the existing works focusing on navigation for visually impaired people.

3. Methodology

The methodology of this footpath detection process consists of the following stages: data collection, data preprocessing, model training, quantization, obstacle distance detection, and finally, the smartphone application implementation. These stages are represented using a block diagram in Fig. 2 and further explained in detail in the later sections.

3.1. Data collection

The dataset has been collected from Kaggle, named “Footpath Image Dataset” [4], which consists of 3000 footpath images and their corresponding masks. These images are mostly taken from the vicinity of Dhaka city, highlighting various footpath conditions, and the walkable

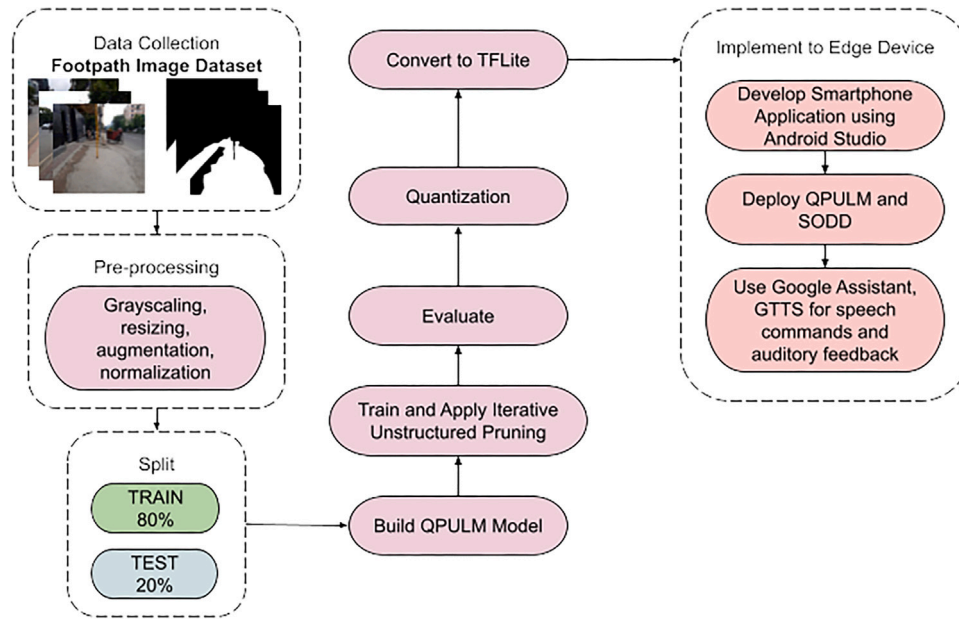


Fig. 2. Proposed methodology block diagram.

areas conformed to the captured images are shown in their respective masks. The masked images are in black and white, where the black segment represents obstacles or non-walkable paths, and the white segment constitutes the walkable path. The black segment considers humans, trees, unwanted litter, trash cans, street side stalls, and anything that blocks the path as an obstacle.

3.2. Data pre-processing

The data from the Footpath Image Dataset were first converted to grayscale images and resized to 256 by 256. Grayscale was considered because of its low computational cost and better performance than RGB images for image segmentation. Data augmentation has been carried out by manipulating the brightness levels of the images ranging from 50% to 100% using ImageDataGenerator. As a result, the volume of images doubled in size, giving enough samples for a larger set to achieve a better learning curve. The augmented images were then normalized. The dataset is split into 80:20 training (with augmented images) and testing sets before being passed to the model for training.

3.3. Model architecture

The proposed model, named 'Quantized and Pruned UNet-based Lightweight MobileNet Model' (QPULM), was built using convolutional neural networks (CNN) for semantic segmentation by defining an encoder and a decoder in a UNet-based architecture, as shown in Fig. 3. UNet is a popular deep learning architecture known for its performance in image segmentation tasks. It was developed by Olaf Ronneberger, Philipp Fischer, and Thomas Brox in 2015 [24], initially for biomedical image segmentation, which is now used for various image data types. In QPULM, the UNet architecture has the encoder consisting of a pre-trained MobileNetV3Small model that downsamples the input images, and the decoder has CNN layers with UpSampling2D layers, which upsamples the images to their original size. After defining the model, a separate function was created to perform unstructured iterative pruning on the model layers to reduce the latency and size of the model.

Table 2

MobileNetV3Small architecture summary.

Input	Operator	Exp size	#out	SE	NL	s
$224^2 \times 3$	conv2d, 3×3	–	16	–	HS	2
$112^2 \times 16$	bneck, 3×3	16	16	✓	RE	2
$56^2 \times 16$	bneck, 3×3	72	24	–	RE	2
$28^2 \times 24$	bneck, 3×3	88	24	–	RE	1
$28^2 \times 24$	bneck, 5×5	96	40	✓	HS	2
$14^2 \times 40$	bneck, 5×5	240	40	✓	HS	1
$14^2 \times 40$	bneck, 5×5	240	40	✓	HS	1
$14^2 \times 40$	bneck, 5×5	120	48	✓	HS	1
$14^2 \times 48$	bneck, 5×5	144	48	✓	HS	1
$14^2 \times 48$	bneck, 5×5	288	96	✓	HS	2
$7^2 \times 96$	bneck, 5×5	576	96	✓	HS	1
$7^2 \times 96$	bneck, 5×5	576	96	✓	HS	1
$7^2 \times 96$	conv2d, 1×1	–	576	✓	HS	1
$7^2 \times 576$	pool, 7×7	–	–	–	–	1
$1^2 \times 576$	conv2d 1×1 , NBN	–	1024	–	HS	1
$1^2 \times 1024$	conv2d 1×1 , NBN	–	k	–	–	1

3.3.1. Training phase

In the QPULM model, a modified U-Net-based architecture features an encoder–decoder structure. U-Net is widely used for semantic segmentation due to its effectiveness, where the goal is to classify each pixel in an input image into a specific class.

MobileNetV3Small is used as the base model for feature extraction. MobileNetV3Small is a lightweight convolutional neural network that is efficient because of its comparatively minimized parameters compared to other pre-trained models while maintaining good accuracy. The 'Small' version is lighter and more compact, making it more deployable to mobile or edge devices. In Table 2, the specifications for MobileNetV3Small [25] are given where SE denotes whether there is a Squeeze-And-Excite in that block, NL denotes the type of nonlinearity used, HS denotes h-swish, RE denotes ReLU, and NBN denotes no batch normalization.

The output from this base model is passed through the expansive path (decoder). The decoder consists of 6 Conv2D layers followed by an UpSampling2D layer (as seen in Eq. (1)). This helps in upsampling the spatial resolution of the feature maps. The number of filters in the Conv2D layers decreases progressively from 512 to 32.

$$\text{Upsampling2D}(x) : \text{Spatially upsample feature map } x \text{ to increase spatial resolution.} \quad (1)$$

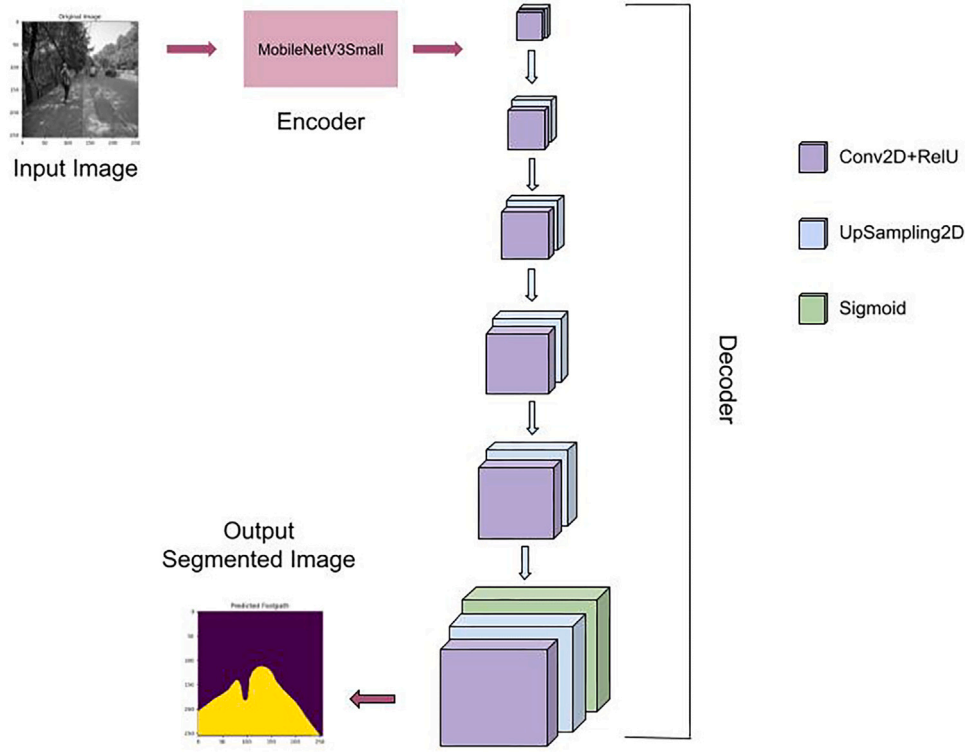


Fig. 3. Encoder-Decoder model architecture.

Finally, the last Conv2D layer outputs a single channel with a sigmoid activation (Eq. (2)), representing the binary segmentation mask.

$$\sigma(x) = \frac{1}{1+e^{-x}} \quad (2)$$

Algorithm 1 MobileNet-Based Segmentation Model

Require: Backbone: MobileNetV3Small, Input image: input_img

Ensure: Segmentation model

1: backbone $\leftarrow$ MobileNetV3Small

Decoder:

```

2:  $x \leftarrow \text{Conv2D}(512, (3, 3), \text{activation} = \text{"relu"}, \text{padding} = \text{"same"})(\text{backbone})$ 
3:  $x \leftarrow \text{UpSampling2D}((2, 2))(x)$ 
4:  $x \leftarrow \text{Conv2D}(256, (3, 3), \text{activation} = \text{"relu"}, \text{padding} = \text{"same"})(x)$ 
5:  $x \leftarrow \text{UpSampling2D}((2, 2))(x)$ 
6:  $x \leftarrow \text{Conv2D}(128, (3, 3), \text{activation} = \text{"relu"}, \text{padding} = \text{"same"})(x)$ 
7:  $x \leftarrow \text{UpSampling2D}((2, 2))(x)$ 
8:  $x \leftarrow \text{Conv2D}(64, (3, 3), \text{activation} = \text{"relu"}, \text{padding} = \text{"same"})(x)$ 
9:  $x \leftarrow \text{UpSampling2D}((2, 2))(x)$ 
10:  $x \leftarrow \text{Conv2D}(32, (3, 3), \text{activation} = \text{"relu"}, \text{padding} = \text{"same"})(x)$ 
11:  $x \leftarrow \text{UpSampling2D}((2, 2))(x)$ 
12:  $x \leftarrow \text{Conv2D}(1, (3, 3), \text{activation} = \text{"sigmoid"}, \text{padding} = \text{"same"})(x)$ 
13:  $x \leftarrow \text{UpSampling2D}(\text{size} = (1, 1))(x)$ 
14: model  $\leftarrow$  Model(inputs=input_img, outputs=x)
15: return model

```

Before the training commences, this neural network is compiled with an optimizer, loss function, and several performance metrics. Adam is employed as the optimizer for its adaptive learning rates and low memory requirements, which also helps the model converge faster.

$$W \leftarrow W - 0.001 \frac{m}{\sqrt{v+\epsilon}} \quad (3)$$

Binary cross-entropy is selected as the loss function, which is highly suitable for binary segmentation tasks.

$$\text{BinaryCrossentropy}(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^N (y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)) \quad (4)$$

Lastly, a set of performance metrics, including accuracy, precision, recall, IoU (Intersection over Union), and dice coefficient, is utilized to assess the model's performance during training.

3.3.2. Unstructured iterative pruning

An unstructured iterative pruning technique (Algorithm 2) is applied to Conv2D and Dense layers in the model, inspired by the previous work done in [26]. Fig. 4 illustrates the entire algorithm, where, at first, the proposed model is trained for 40 epochs with Early Stopping (patience = 15) and ReduceLROnPlateau scheduler callbacks. After training, the model passes through the unstructured pruning step to identify and remove less salient connections without enforcing a specific pattern (e.g., entire rows or columns) and is retrained with the active weights, repeating the cycle for three complete iterations.

The algorithm utilizes *prune_low_magnitude* function from the TensorFlow Model Optimization Toolkit, which computes the absolute values of all weights in each layer and sorts them by magnitude before removing a proportion of salient connections determined by the sparsity level (as shown in Eq. (5)). The pruning is guided by a constant sparsity schedule (Eq. (6)), maintaining a sparsity level of 50% across layers. This ensures that half of the weights in the pruned layers are set to zero.

$$\text{PrunedWeights}(W, M) = W \odot M, \quad \text{where } M \in \{0, 1\}^{n \times m}. \quad (5)$$

where W is the original weights of a neural network layer, M is the pruning mask applied to the weights, $M \in \{0, 1\}^{n \times m}$, and $\odot$ is the element-wise multiplication (Hadamard product).

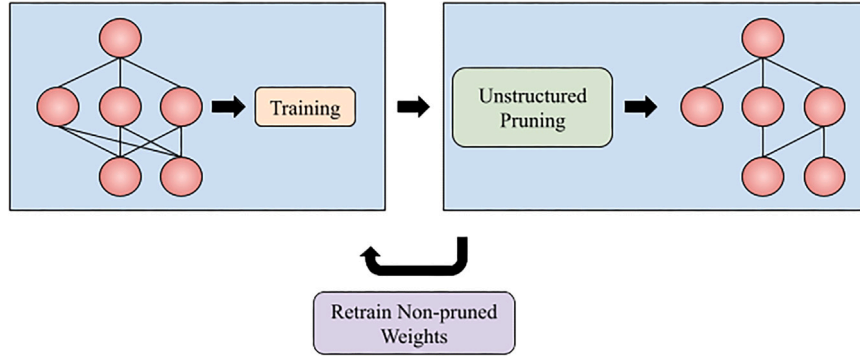


Fig. 4. Graphical representation of unstructured iterative pruning.

Algorithm 2 Unstructured Iterative Pruning**Require:** *model, X_train, y_train, X_valid, y_valid, iterations***Ensure:** Pruned model

```

1: for  $i \leftarrow 1$  to iterations do
2:   Compile the model with optimizer=Adam(), loss="binary_crossentropy"
3:   Train the model
4:   Prune the model:
5:   for each layer  $l$  in model.layers do
6:     if ( $l$  is instance of Conv2D or Dense) then
7:       pruning_schedule  $\leftarrow$  pruning_schedule.ConstantSparsity(0.5, 0, 1)
8:       pruned_layer  $\leftarrow$  prune.prune_low_magnitude( $l$ , pruning_schedule)
9:       pruned_layer.build( $l$ .input_shape)
10:      model.get_layer(name= $l$ .name).set_weights(pruned_layer.get_weights())
11:      model.get_layer(name= $l$ .name).trainable  $\leftarrow$  False
12:    end if
13:  end for
14: end for

```

The masks M generated (Eq. (5)) by the pruning algorithm is either 0 or 1 depending on whether the weights W are pruned or kept.

$$\text{ConstantSparsity}(s, b, e) = \begin{cases} s & \text{for } b \leq t \leq e, \\ 0 & \text{otherwise.} \end{cases} \quad (6)$$

where s is the desired sparsity level (constant), b is the beginning step of pruning, e is the ending step of pruning, and t is the current training step.

3.4. Quantization

Quantization is a technique employed to make the deep learning model more lightweight by reducing the memory footprint of the model parameters. The trained Keras model is quantized using TensorFlow Lite (TFLite), which converts the model's original 32-bit-floating-point weights and activations to a lower precision, 8-bit integers. TFLite, introduced by Google AI Edge, offers model optimization by quantization, pruning, and clustering. It helps reduce model size and latency and is suitable for edge devices with limited memory or computational power. This conversion drastically reduces the model's size (about four times) without significantly compromising the model's accuracy. The original model had a cumulative accuracy of approximately 97% with a size of 20 MB and, after conversion to TFLite through quantization, produced about 99% with a size of 5 MB. This quantized model is now easily deployable to edge devices, such as a smartphone application, for this case.

3.5. Segmentation-based obstacle distance detection (SODD)

The proposed technique, "Segmentation based Obstacle Distance Detection" (SODD), is used to detect obstacles and their probable distance area from the first person point of view. Firstly, the predicted

binary mask is divided into a grid of 9 segments. For each grid segment, the pixels representing the walkable area are counted to estimate the probable area distance until the next obstacle or non-walkable area is encountered. The area estimation is computed for going towards the left, right, and forward directions (as shown in Eqs. (7)–(10)).

$$\alpha = \frac{\text{original width } (ft)}{\text{image width}} \quad (7)$$

$$\text{predicted segment width } (ft) = \alpha \times \text{pixel count in width} \quad (8)$$

$$\beta = \frac{\text{original height } (ft)}{\text{image height}} \quad (9)$$

$$\text{predicted segment height } (ft) = \beta \times \text{pixel count in height} \quad (10)$$

According to experimental analysis, obstacle distance calculation depends on several parameters, such as camera positioning when taking footpath pictures and camera resolution. The distance between the test subject and the camera reference point was measured in real-time to estimate the vertical distance. Similarly, the horizontal footpath distance encompassed in the image was also measured. In this experiment, α (7) and β (9) constant values (conversion factors) were computed separately because, in real-life applications, vertical and horizontal distances captured in images are comparatively different based on the camera resolution dependency. To improve the precision and reduce uncertainty, several such experiments were conducted using multiple smartphone cameras to get the estimated values of α and β , which are 0.023 and 0.11, respectively.

The predicted masked image was divided into a grid of 5:9 (Fig. 5), where each segment's pixel count was calculated (as shown in Algorithm 3). As the picture was divided into five rows, the predicted

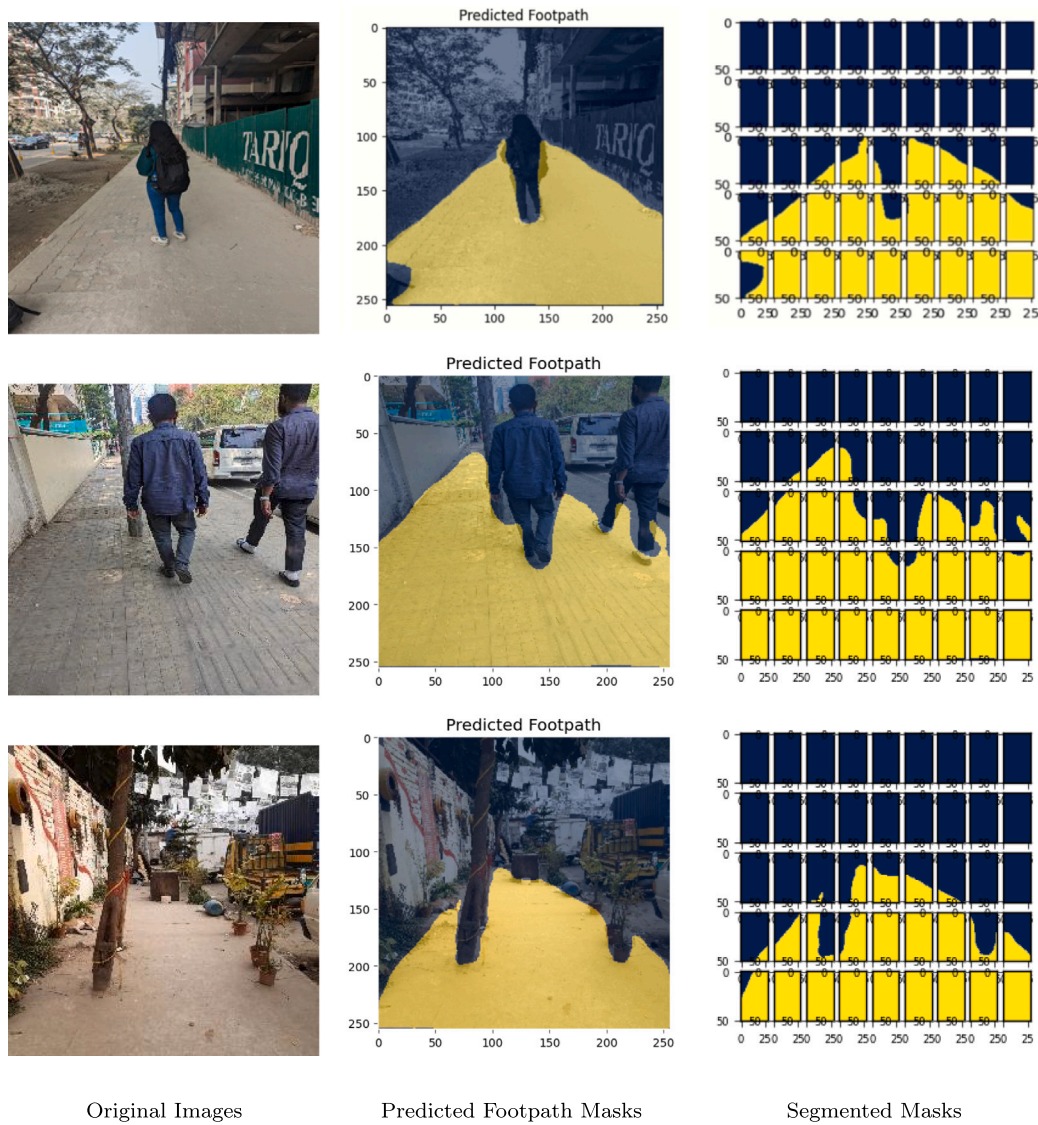


Fig. 5. Mask segmentation.

Algorithm 3 SODD**Require:** Predicted Masked Image**Ensure:** Predicted Distance

- 1: Split image into a 5x9 grid
- 2: Calculate masked pixel count in each segment:
 $\text{masked_pixel_segment} \leftarrow [\text{np.sum}(\text{segment}) \text{ for segment in segments}]$
- 3: Iterate over grid rows and nested columns
- 4: Calculate front, left, and right segments' pixel count
- 5: Compare front, left, and right segments with a fully masked segment:
 $\text{pixel count} \geq \text{fully masked segment}$
 $\text{horizontal distance} += \text{predicted segment width}$
 $\text{vertical distance} += \text{predicted segment height}$

segment height engendered a value of 5.42 ft, and nine columns resulted in a value of 0.66 ft for segment width. From the base position of the picture, the segments are clustered into three for each of the left, right, and front directions. Based on many measurements, on average, a person takes 2 ft to move properly in any direction. In SODD, the probable walkable path is depicted on the front, left, and right segments, which is calculated up until the segment an obstacle is found (as shown in Fig. 6).

3.6. Smartphone implementation

An Android application has been developed using Chaquopy in Android Studio. Chaquopy is a framework that allows users to run Python scripts in Java/Kotlin code in Android Studio. This made it easier to implement the model and obstacle detection methods. During inferencing, some frames were skipping, for which threads were utilized to perform the tasks, image preprocessing, post-processing, and

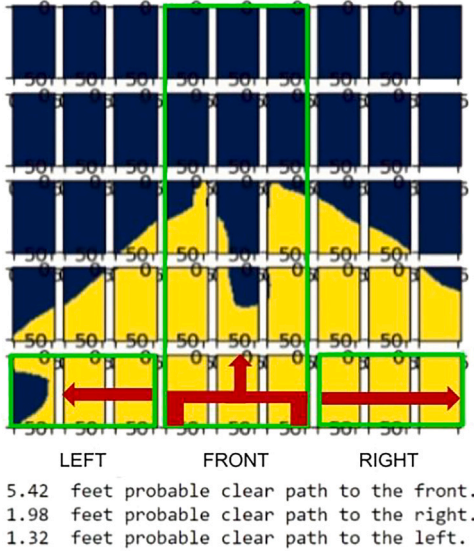


Fig. 6. Obstacle distance prediction.



Fig. 7. Application implementation.

SODD assigned in the background. The TFLite version of the QPULM model has been deployed in the application. To apply the footpath and obstacle detection, one can capture a live image of the footpath in front of them or simply choose recent photos from the gallery. As this application has been designed for blind people, with the help of Google Assistant, one can access the application through voice, where certain commands can be utilized to perform the methods mentioned above. After taking an image, Google text-to-speech (GTTS) API is used, which tells the estimated probable clear path to the front, right, and left. An example can be seen in Fig. 7, where an image is captured using the smartphone camera, and then the resulting output of probable clear paths is shown on screen and delivered via audio.

4. Results and evaluation

The model is evaluated based on certain relevant parameters and metrics discussed below.

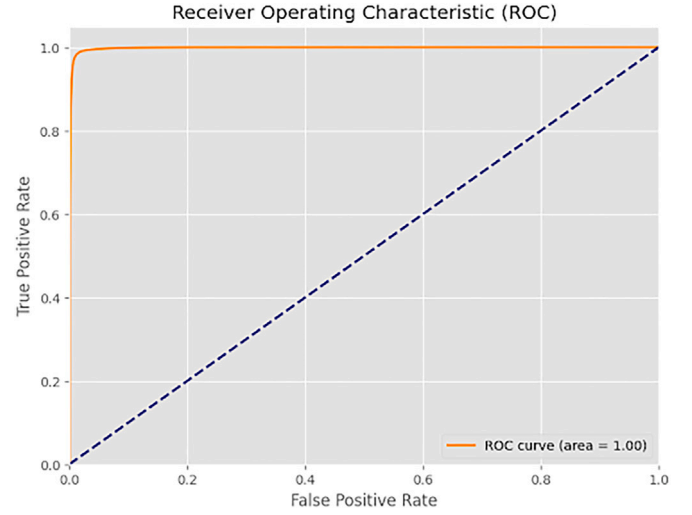


Fig. 8. AUC-ROC Curve.

4.1. Evaluation metrics

Several pre-trained models were implemented using transfer learning to analyze and compare the performance of such models for footpath image segmentation. The models, namely InceptionV3-based Unet, ResNet50V2-based Unet, MobileNet2-based Unet, MobileNetV3, and QPULM, were evaluated using several performance evaluation metrics on the unseen test dataset. These metrics include accuracy, precision, recall, IoU (Intersection over Union), and dice coefficient.

Accuracy measures the overall correctness of the model's predictions, calculated as the ratio of correctly predicted instances to the total instances.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (11)$$

Precision gauges the accuracy of positive predictions, indicating the proportion of correctly predicted positive instances among all instances predicted as positive.

$$\text{Precision} = \frac{TP}{(TP + FP)} \quad (12)$$

Recall (Sensitivity) assesses the model's ability to capture all positive instances, calculated as the ratio of correctly predicted positive instances to the total actual positive instances.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (13)$$

IoU (Intersection over Union) measures the overlap between the predicted and actual regions, providing an assessment of spatial accuracy.

$$\text{IoU} = \frac{TP}{TP + FP + FN} \quad (14)$$

The Dice Coefficient quantifies the spatial similarity between the predicted and the ground truth regions, emphasizing their overlap.

$$\text{Dice} = \frac{2 \times TP}{2 \times TP + FP + FN} \quad (15)$$

TP, TN, FP , and FN are the true positive, true negative, false positive, and false negative of the dataset. These measures represent different classifications based on comparing predicted and actual outcomes.

ROC stands for receiver operating characteristic, a graph-based performance measurement technique for classification at various thresholds that compares the model's performance by plotting the true positive rate (TPR) vs. the false positive rate (FPR) at different classification thresholds. If the area under the ROC curve (AUC) is closer to 1, then

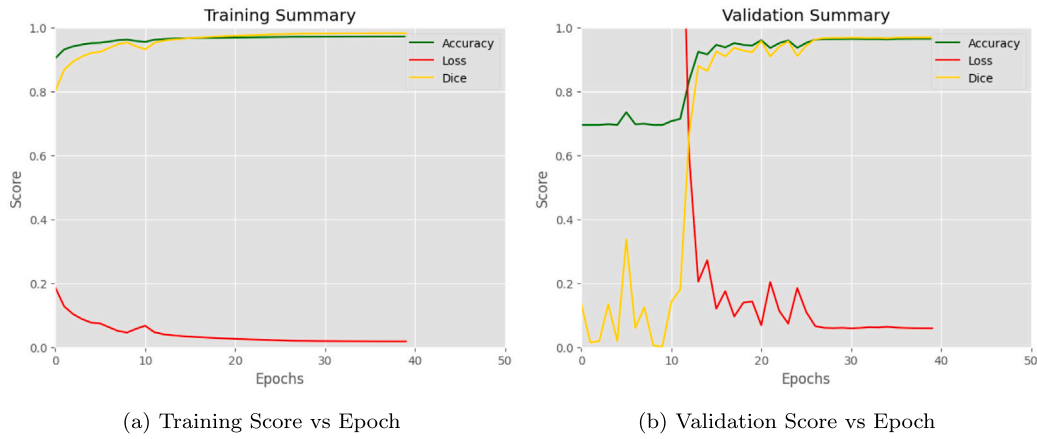


Fig. 9. Training and validation accuracy, Loss and Dice.

Table 3
Comparison analysis between prediction models.

Model	Accuracy	Precision	Dice	IoU	Model size
<i>InceptionV3_Unet</i>	80.80%	0.65	0.57	0.40	250 MB
<i>ResNet50V2_Unet</i>	85.68%	0.73	0.69	0.52	89.7 MB
<i>MobileNetV3</i>	93.65%	0.93	0.92	0.85	60 MB
<i>MobileNetV2_Unet</i>	91.85%	0.88	0.82	0.70	26 MB
<i>QPULM</i>	99.37%	0.98	0.97	0.94	5 MB

Table 4
Comparison analysis between TFLite prediction models.

Model	TFLite accuracy	Execution time
<i>ResNet50V2_Unet</i>	87.16%	154.4 ms
<i>MobileNetV3</i>	95.36%	122.2 ms
<i>MobileNetV2_Unet</i>	93.45%	122.2 ms
<i>QPULM</i>	99.37%	120.7 ms

the model's performance would be greater. Based on this analysis, QPULM showed a perfect fit with an AUC score of 1, as shown in the Fig. 8.

In Fig. 9, for both training and validation, accuracy, loss, and dice were presented for each epoch. In Fig. 9(a), both accuracy and dice coefficient graphs are upward-sloping, which means with each epoch, the model's training performance has substantially improved over time. Moreover, the loss is downward sloping, highlighting that the difference between prediction and target values is minimized with each epoch. Similarly, in the Fig. 9(b), all the performance metrics resembled the training summary, further solidifying the model's performance.

Table 3 compared the models using accuracy, precision, dice coefficient, IoU, and the model size. From this table, it can be observed that among the pre-trained models, MobileNetV3 has the better performance. Overall, the proposed QPULM model is the best-performing model (QPULM), showing an accuracy of 99.37%, and is the most compact and optimized based on its comparatively smaller size.

In Fig. 10, the aforementioned pre-trained models' accuracy and loss have been displayed as a bar chart, where the proposed model showed the highest accuracy and lowest loss.

In Table 4, the three best-performing pre-trained models from Table 3 have been converted to lightweight TFLite versions, which were further compared in terms of accuracy and execution time against the QPULM model. The proposed model showed better accuracy, considering its size and execution time.

In Fig. 11, the QPULM model's before and after quantization results are visualized. Fig. 11(a) shows the execution time decreasing from 647 ms to 120.7 ms. Moreover, in Fig. 11(b), the model's size before quantization is 27.4 MB, whereas, after quantization, it is reduced to almost five times its original size, that is, 5.11 MB.

Accuracy and Loss Bar Chart

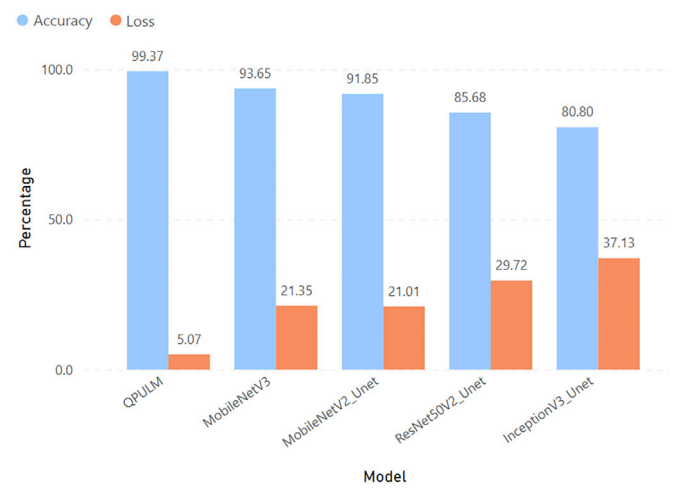


Fig. 10. Accuracy loss bar chart.

Table 5 summarizes some existing similar works that have implemented sophisticated deep learning techniques and integrated them into edge devices. However, some did not properly evaluate their system based on accuracy, precision, recall, and ROC. The SSDLite MobileNetV2 model [18] achieved satisfactory accuracy, precision, and recall. The other models trained on different datasets have metrics such as IOU and mAP that are not as satisfactory. However, the proposed model stands out in this comparison regarding the given performance metrics and accessible system implementation.

5. Conclusion

Footpaths in Bangladesh are challenging to navigate, especially for the visually impaired. The growing evolution of machine learning has engendered many optimum solutions for such issues, one of which is the proposed research, which is the current state of the art. This research proposes a system for the visually impaired to quickly identify walkable footpaths and detect the nearest obstacle using a smartphone device. Portability and accessibility of the edge device were considered, and a lightweight, optimized model, QPULM, was trained and integrated into a smartphone as an Android application. An Android application was developed in Android Studio using Chaquopy, where the QPULM model is used to detect walkable footpaths, and the SODD algorithm is used to detect the distance of the object present in the

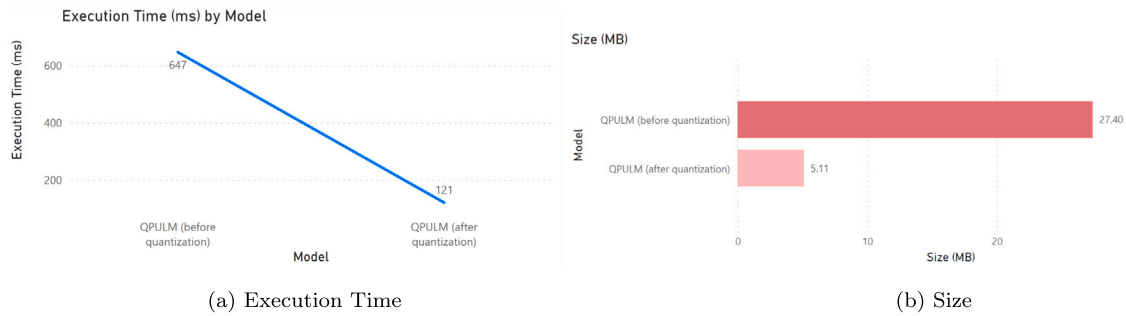


Fig. 11. QPULM before and after quantization.

Table 5

Comparison analysis between existing literature.

Ref	Primary features	System used	Model	Performance metrics
[16]	Road segmentation & Object detection	OAK-D	SSD MobileNet	Acc = 90.1%, IOU = 84.4%
[17]	Pothole detection	Raspberry pi	MobileNet SSD	Acc = 72%, mAP = 28.2%
[18]	Object and surrounding atmosphere detection	Raspberry pi 4B	SSDLite MobileNetV2	Acc = 88.89%, Precision = 0.892, Recall = 0.89
Proposed model	Walkable footpath detection	Smartphone	QPULM	Acc = 99.37%, Precision = 0.98, Dice = 0.97

footpath towards left, right, and front through image segmentation by taking an image of the path in front of the person.

It is important to note that the current system has limitations. For instance, images captured at night may not yield the same results as those taken during the day. The lack of clarity in these images, caused by the dark surroundings, can make it challenging to identify footpaths. Moreover, the dataset used for training only represents the footpaths in Bangladesh and does not fully capture real-world scenarios and conditions such as weather and varying lighting.

In the future, this work can be further extended by incorporating an extended dataset that captures scenes from different countries and conditions to enhance generalization. Ablation studies can be conducted to compare the performance of different DNNs, such as Transformers and Graph Neural Networks. Furthermore, a 360-degree video analysis system can be developed using multimodal data such as audio and video to provide comprehensive information about the user's surroundings. This work could potentially enhance the footpath navigation system by incorporating more detailed information about the vicinity, further improving its performance.

CRedit authorship contribution statement

Md. Ishan Arefin Hossain: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Jareen Anjom:** Writing – original draft, Visualization, Software, Methodology, Formal analysis, Data curation. **Rashik Iram Chowdhury:** Writing – original draft, Visualization, Software, Methodology, Formal analysis, Data curation.

Funding

This research has been funded by the North South University Conference Travel and Research Grant (CTRG) 2023–2024 under the project ID: CTRG-23-SEPS-29.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data that support the findings of this study are available from the corresponding author upon reasonable request.

References

- [1] WHO. WHO blindness and vision impairment. 2024, <https://www.who.int/news-room/fact-sheets/detail/blindness-and-visual-impairment>, [Accessed January 2024].
- [2] Countries by population density. 2024, <https://worldpopulationreview.com/country-rankings/countries-by-density>, [Accessed January 2024].
- [3] Staff Correspondent. Six million Bangladeshis visually impaired: speakers. 2024, <https://www.thedailystar.net/city/news/six-million-bangladeshis-visually-impaired-speakers-1856473>, [Accessed January 2024].
- [4] AFIF AL MAMUN. Footpath image dataset. 2024, <https://www.kaggle.com/datasets/affaniks/footpath-image-dataset/>, [Accessed January 2024] [dataset].
- [5] Datta A, Meghla TI, Khatun T, Bhuiya MH, Shuvo SR, Rahman Md M. Road object detection in Bangladesh using faster R-CNN: A deep learning approach. In: 2020 IEEE international women in engineering (WIE) conference on electrical and computer engineering (WIECON-ECE) 348–351. 2020, <http://dx.doi.org/10.1109/wiecon-ece52138.2020.9397954>.
- [6] Ng HR, Zhong X, Nam Y, Youn J-H. Deep-learning-based approach for automated detection of irregular walking surfaces for walkability assessment with wearable sensor. Appl Sci 2023;13(13053). <http://dx.doi.org/10.3390/app132413053>.
- [7] Swathi K, Vamsi B, Rao NT. A deep learning-based object detection system for blind people. Lecture notes in networks and systems., 2021, p. 223–31. http://dx.doi.org/10.1007/978-981-16-1773-7_18.
- [8] Kumar N, Jain A. A deep learning based model to assist blind people in their navigation. J Inf Technol Educ Innov Pr 2022;21:095–114. <http://dx.doi.org/10.28945/5006>.
- [9] Sophia B, Chitra D. Segmentation based real time anomaly detection and tracking model for pedestrian walkways. Intell Autom & Soft Comput 2023;36:2491–504. <http://dx.doi.org/10.32604/iasc.2023.029799>.
- [10] Alsolai H, Al-Wesabi FN, Motwakel A, Drar S. Assisting visually impaired people using deep learning-based anomaly detection in pedestrian walkways for intelligent transportation systems on remote sensing images. Deleted J 2. 2023. <http://dx.doi.org/10.57197/jdr-2023-0021>.
- [11] Pustokhina IV, Pustokhin DA, Vaiyapuri T, Gupta D, Kumar S, Shankar K. An automated deep learning based anomaly detection in pedestrian walkways for vulnerable road users safety. Saf Sci 2021;142:105356. <http://dx.doi.org/10.1016/j.ssci.2021.105356>.
- [12] Alohal MA, Aljebreen M, Nemri N, Allafi R, Duhayyim MA, Alsaid MI, Al-neil AA, Osman AE. Anomaly detection in pedestrian walkways for intelligent transportation system using federated learning and harris hawks optimizer on remote sensing images. Remote Sens 2023;15(3092). <http://dx.doi.org/10.3390/rs15123092>.
- [13] Hussan MIT, Saidulu D, Anitha PT, Manikandan A, Naresh P. Object detection and recognition in real time using deep learning for visually impaired people. Int J Electr Electron Res 2022;10:80–6. <http://dx.doi.org/10.37391/ijeer.100205>.

- [14] Okolo GI, Althobaiti T, Ramzan N. Smart assistive navigation system for visually impaired people. *JDR* 2025;4(1). <http://dx.doi.org/10.57197/JDR-2024-0086>.
- [15] Morra D, Zhu X, Liu C, Fu K, Duarte F, Mora S, He Z, Ratti C. Mapping sidewalk accessibility with smartphone imagery and visual AI: a participatory approach. *Philos Trans R Soc A Math Phys Eng Sci* 2024;382. <http://dx.doi.org/10.1098/rsta.2024.0106>.
- [16] Mahendran JK, Barry DT, Nivedha AK, Bhandarkar SM. Computer vision-based assistance system for the visually impaired using mobile edge artificial intelligence. In: 2022 IEEE/CVF conference on computer vision and pattern recognition workshops. *CVPRW*, Vol. 241, 2021, p. 8–2427. <http://dx.doi.org/10.1109/cvprw53098.2021.00274>.
- [17] V.K N, Ullal BS, Gogoi P, Singh K, Biswas A, Yash K. Smart navigation aid for visually impaired person using a deep learning model. In: Third International Conference on Artificial Intelligence and Smart Energy. *ICAIS*, vol. 104, 2023, p. 9–1053. <http://dx.doi.org/10.1109/icaais56108.2023.10073922>.
- [18] Islam RB, Akhter S, Iqbal F, Rahman Md SU, Khan R. Deep learning based object detection and surrounding environment description for visually impaired people. *Heliyon* 2023;9:e16924. <http://dx.doi.org/10.1016/j.heliyon.2023.e16924>.
- [19] Lima R, Barreto L, Amaral A, Paiva S. Visually impaired people positioning assistance system using artificial intelligence. *IEEE Sensors J* 2023;23:7758–65. <http://dx.doi.org/10.1109/jsen.2023.3244128>.
- [20] Suresh M, Shaik AS, Premalatha B, Narayana VA, Ghinea G. Intelligent & smart navigation system for visually impaired friends. In: *Communications in computer and information science*. vol. 37, 2023, p. 4–383. http://dx.doi.org/10.1007/978-3-031-35641-4_30.
- [21] Sajini S, Pushpa B. Sensor enabled proximity detection with hybridisation of IoT and computer vision models to assist the visually impaired. *Eng Technol Appl Sci Res* 2023;13:12284–8. <http://dx.doi.org/10.48084/etasr.6410>.
- [22] Merugu S, Jain K, Mittal A, Raman B. Sub-scene target detection and recognition using deep learning convolution neural networks. *Lecture notes in electrical engineering*, vol. 108, 2020, p. 2–1101. http://dx.doi.org/10.1007/978-981-15-1420-3_119.
- [23] Suresh M, Shaik AS, Premalatha B, Narayana VA, Ghinea G. Intelligent & smart navigation system for visually impaired friends. In: *Communications in computer and information science*. vol. 37, 2023, p. 4–383. http://dx.doi.org/10.1007/978-3-031-35641-4_30.
- [24] Ronneberger O, Fischer P, Brox T. U-NET: Convolutional networks for biomedical image segmentation. *lecture notes in computer science*, vol. 23, 2015, p. 4–241. http://dx.doi.org/10.1007/978-3-319-24574-4_28.
- [25] Howard A, et al. Searching for MobileNetV3. In: 2019 IEEE/CVF international conference on computer vision. *ICCV*, Seoul, Korea (South); 2019, p. 1314–24. <http://dx.doi.org/10.1109/ICCV.2019.00140>.
- [26] Han S, Pool J, Tran J, Dally WJ. Learning both weights and connections for efficient neural networks. In: *Advances in neural information processing systems* 28. *NIPS* 2015, 2015, p. 1135–43, Available at: <https://proceedings.neurips.cc/paper/2015/file/ae0eb3eed39d2bcef4622b2499a05fe6-Paper.pdf>.